Creating and Deploying Production-Ready Agentic Systems

Dimitris Tsirmpas

Athens University of Economics and Business

Archimedes, Athena Research Center

June 20, 2026

What we will be talking about

1. The LLM as a System Component
   - Inherent LLM limitations
   - What do LLMs bring to the table?
   - When to use LLMs?
2. Tool-Based Reasoning using ReAct
   - LLM tool-calling
   - Thought, Action, Observation
   - How much autonomy should we give the agent?
3. Agent-to-Agent Interaction
   - Single-Agent vs. Multi-Agent
   - Orchestrating Interaction
   - What do we actually need?

Section 1

The LLM as a System Component

Subsection 1

Inherent LLM limitations

Section 1

The LLM as a System Component

Subsection 2

What do LLMs bring to the table?

Section 1

The LLM as a System Component

Subsection 3

When to use LLMs?

LLMs are not just task-specific tools

- So far, we have used LLMs as specialized tools for specific tasks.
- While useful, these tasks can generally be accomplished by simpler and more scalable models (with some performance loss).
- Where LLMs differ, is that they are autonomous, which allows for previously impossible use-cases.

When you are holding a hammer...

- Suppose you are given the following task, with appropriate resources from management.

Use case

Find all reviews that concern competitors of our product, and tell us what percentage of them mention our product.

When you are holding a hammer...

Our problem could be decomposed in three parts

1. Discovery: Find pages containing reviews.
2. Filtering: Find the HTML sections containing the reviews.
3. Analysis: Find which reviews reference the product.

Which of these must necessarily be solved with LLMs? Which can be solved with traditional ways? What trade-offs are we making in the latter case?

... you treat everything as if it were a nail

The easy solution

We instruct an LLM to retrieve reviews via RAG. We isolate the reviews using prompting, then instruct the LLM to find whether our product is mentioned.

Result

Code is deployed within the day. Management is billed 50,000€ in a week.

But maybe we can do better

The system solution

We use a traditional scrapper to find relevant reviews. For each page, we instruct the LLM to extract the reviews. We then run `reviews.str.contains(product_name).sum()`.

Result

Code is deployed within two days. Management is billed 20€ in a week.

Section 2

Tool-Based Reasoning using ReAct

Subsection 2

Thought, Action, Observation

What is ReAct?

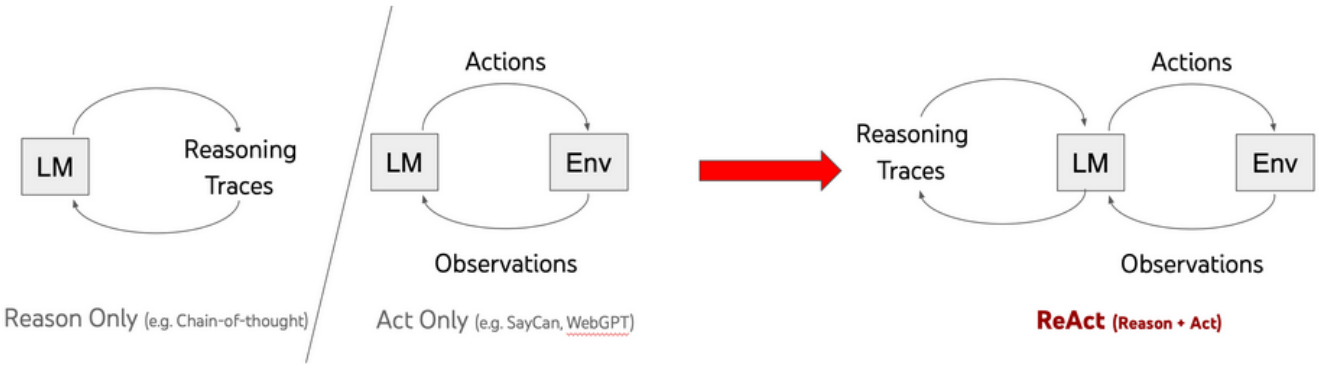


Figure: https://arxiv.org/abs/2210.03629

Definition

ReAct: A technique that interleaves reasoning steps ("Thought") with Actions and Observations to guide an LLM through planning and tool use.

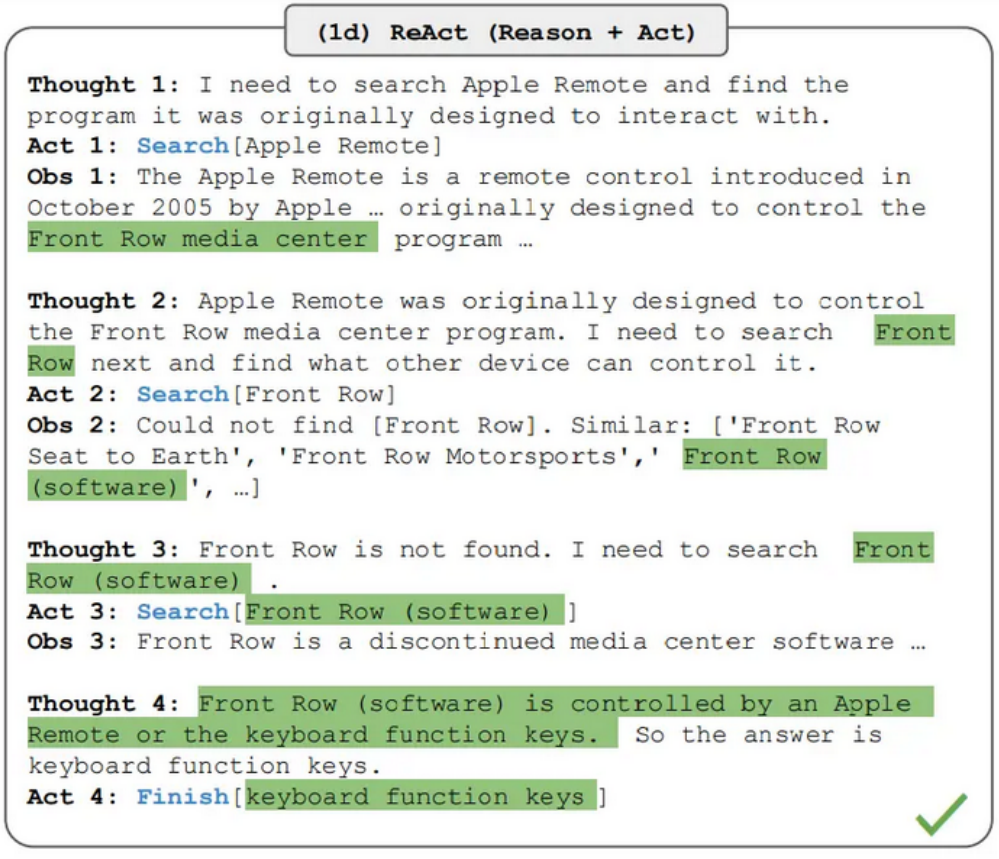
ReAct: Tool-Calling

Tool example

```python
from smolagents import tool
@tool
def calculator(expr: str) -> float:
  """
  Evaluate a basic arithmetic expression.

  Args:
    expr: The math expression to evaluate, such as "2
                                + 3 * (4 / 2)".
  """
  return eval(expr, {"__builtins__": None}, {})
```

High-level Concept



ReAct loop

- Create a plan.
- Identify action using reasoning (CoT).
- Take action.
- Observe result and update plan.

Figure: https://arxiv.org/abs/2210.03629

ReAct: Early examples

You are an agent that answers questions by using two actions:
- search[query]: Google search the query. You can also use it to calculate math.
- finish[answer]: return the answer

Your generation should have the following format:
Thought: to analyze the observation and inform the action.
Action: your action

Question: If I have seven trillion dollars today, can I buy Apple, Nvidia, and Microsoft? If not, how much more money do I need?

Figure: https://rdi.berkeley.edu/llm-agents/f24

Thought: I need to find the current market capitalization of Apple, Nvidia, and Microsoft to understand how much it would cost to buy all of them.
Action: search["current market capitalization of Apple, Nvidia, and Microsoft"]

Observation: "Nvidia's market cap stood at $1.832 trillion as of about 10 a.m. ET Wednesday, while Alphabet's was at $1.814 trillion, bringing Nvidia behind just Microsoft (MSFT) at $3.015 trillion and Apple (AAPL) at $2.836 trillion."

Thought: Now that I have the market cap for all companies, let's add them together and see if $7 trillion is enough.
Action: search["1.832 trillion + 3.015 trillion + 2.836 trillion"]

Observation: 7683000000000

Thought: The amount of money needed in addition to the $7 trillion is $683 billion.
Action: finish["You need an additional $683 billion to be able to buy Apple, Nvidia, and Microsoft."]

Figure: https://rdi.berkeley.edu/llm-agents/f24

Section 2

Tool-Based Reasoning using ReAct

Subsection 3

How much autonomy should we give the agent?

Agency levels

Agency Level	Description	Short name	Example Code
☆☆☆	LLM output has no impact on program flow	Simple processor	`process_llm_output(llm_response)`
★☆☆	LLM output controls an if/else switch	Router	`if llm_decision(): path_a() else: path_b()`
★★☆	LLM output controls function execution	Tool call	`run_function(llm_chosen_tool, llm_chosen_args)`
★★☆	LLM output controls iteration and program continuation	Multi-step Agent	`while llm_should_continue(): execute_next_step()`
★★★	One agentic workflow can start another agentic workflow	Multi-Agent	`if llm_trigger(): execute_agent()`
★★★	LLM acts in code, can define its own tools / start other agents	Code Agents	`def custom_tool(args): ...`

Figure: https://huggingface.co/docs/smolagents/en/conceptual_guides/intro_agents

Section 3

Agent-to-Agent Interaction

Subsection 1

Single-Agent vs. Multi-Agent

Section 3

Agent-to-Agent Interaction

Subsection 2

Orchestrating Interaction

Section 3

Agent-to-Agent Interaction

Subsection 3

What do we actually need?